More practice - Stacks and queues

3 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

3
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Stacks and queues | 3 questions | tap a title to jump to it

M	1. Implement Python's itertools.tee	1Point3Acres	2026-08-20
M	2. Nested Function Expression Evaluator	1Point3Acres	2026-06-10
M	3. Find longest increasing contiguous subarray	PracHub	2026-01-06

STUDY PATH: question 144 of 167 (more practice)

Priority score 3.5 | Stacks and queues | asked 61 days ago | frequency Single report

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 1 of 3

Implement Python's itertools.tee

ASKED 2026-08-20 | 61 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 225/228**

ABOUT THIS QUESTION

Topic: More practice - Stacks and queues

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-20 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/python-tee-independent-iterators

QUESTION

Implement the behavior of Python's **`tee`**: accept one iterator and an integer **n**, then return **n** independent iterators over the same source sequence. Define the required function and classes yourself and improve a naive design toward an efficient shared-state implementation.SWEiteratorqueuedata-structureobject-designmediumFrequencySingle reportLast asked2026-08-20Stageonsite-coding

APPROACH

Implement a custom Python iterator class from memory, including __iter__ and __next__.
Trace two and three consumers advancing at different speeds and identify what state must remain available.
Compare a fully materialized baseline with a lazy shared-state design and explain the memory behavior.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the source iterator can be consumed by one consumer first, what happens if one consumer is far ahead, and whether thread safety is needed.

BRUTE FORCE TO SAY OUT LOUD

Store the whole source in a list and give each consumer an index. Simple, but defeats the purpose for large or infinite iterators.

OPTIMAL IDEA

Keep a deque per consumer. When a consumer needs an item and its deque is empty, pull the next item from the source and append it to every consumer's deque. Each item is held only until all consumers have read it. Build with a shared state object and a small iterator class with __iter__ and __next__. The linked-list variant uses one chain of nodes with each consumer holding a pointer, which frees nodes as the slowest consumer moves.

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

Two consumers at different speeds, n = 1, n = 0, an empty source, an exhausted source.

LIKELY FOLLOW-UPS

Thread safety (a lock around the pull), and memory growth if one consumer stalls.

PITFALLS

Pulling from the source once per consumer instead of once per item. Not appending to the other deques.

HOW TO THINK ABOUT IT

The source can only be consumed once, so someone must remember items until every consumer has read them. The state to keep is exactly 'items the slowest consumer has not read yet', so either give each consumer a queue that the leader fills, or keep one shared linked list with a pointer per consumer so memory is freed automatically.

Basis: The itertools.tee documentation's recipe; the linked-list variant is how CPython shares buffers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def tee(source, n):
    items = list(source)  # read it all now
    def reader():
        for x in items:
            yield x
    return [reader() for _ in range(n)]
```

TIME	SPACE
O(n) up front	**O(n)**

WHY IT IS SLOW

It reads the source completely before anything is consumed, never works on an infinite or lazy source, and keeps every item in memory. One buffer per consumer (or a shared linked list) holds only what the slowest consumer has not read.

OPTIMAL SOLUTION · One deque per consumer, filled lazily

TIME	SPACE
O(n) total, O(1) amortized per item per consumer	**O(lag between fastest and slowest consumer)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Memory grows when one consumer stalls. Is there a way to free items automatically?

Use one shared singly linked list with a pointer per consumer. The only references to old nodes are the consumers' pointers, so Python's garbage collector frees nodes behind the slowest consumer without any bookkeeping. This is the design CPython uses.

FOLLOW-UP 2 Make it safe for several threads.

Pull from the source under one lock and append to the queues inside it. Without the lock two consumers can pull two items and append them in different orders to different queues. **Cost:** one lock acquisition per item pulled, O(1) extra per item; contention grows with the number of consumers.

FOLLOW-UP 3 Why not just copy the iterator?

Most iterators (files, generators, sockets) cannot be copied. tee gives independent views by buffering, which is the only thing you can do when the source cannot rewind.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement Python's itertools.tee
#
# Implement the behavior of Python's tee: accept one
# iterator and an integer n, then return n independent
# iterators over the same source sequence. Define the
# required function and classes yourself and improve a
# naive design toward an efficient shared-state
# implementation.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Memory grows when one consumer stalls. Is there a way
# to free items automatically?
# 2. Make it safe for several threads.
# 3. Why not just copy the iterator?
#
# INTERVIEW NOTES
# The source can only be consumed once, so someone must
# remember items until every consumer has read them. The
# state to keep is exactly 'items the slowest consumer has
# not read yet', so either give each consumer a queue that
# the leader fills, or keep one shared linked list with a
# pointer per consumer so memory is freed automatically.
#
# BRUTE FORCE (say it first): Materialize the whole source
# into a list
# Read everything into a list and give each consumer an
# index. Correct for finite data, but it consumes the
# source eagerly, never works on an infinite iterator and
# holds all items in memory. Time O(n) up front. Space
# O(n).


import threading
from collections import deque


# FIRST IDEA (simplest): copy into a list
def tee_naive(iterable, n=2):
    # simplest idea: copy everything into a list
    items = list(iterable)
    # but it never works for endless iterators
    return tuple(iter(items) for _ in range(n))


# BASE QUESTION (optimal): One deque per consumer, filled
# lazily
# Idea: When a consumer's deque is empty it pulls the next
# item from the source and appends it to every consumer's
# deque, then pops its own.
def tee_queues(iterable, n=2):
    source = iter(iterable)
    # one waiting line of values per consumer
    queues = [deque() for _ in range(n)]

    def consumer(mine):
        while True:
            if not mine:
                try:
                    value = next(source)
                except StopIteration:
                    return
                # give the new value to everybody
                for q in queues:
                    q.append(value)
            yield mine.popleft()

    return tuple(consumer(q) for q in queues)


# FOLLOW-UP 1: Memory grows when one consumer stalls. Is
# there a way to free items automatically?
# Answer: Use one shared singly linked list with a pointer
# per consumer. The only references to old nodes are the
# consumers' pointers, so Python's garbage collector frees
# nodes behind the slowest consumer without any
# bookkeeping.
class Cell:
    # one link of the shared list
    def __init__(self):
        self.filled = False
        self.value = None
        self.next = None


# FOLLOW-UP 2: Make it safe for several threads.
# Answer: Pull from the source under one lock and append
# to the queues inside it. Without the lock two consumers
# can pull two items and append them in different orders
# to different queues.
def tee(iterable, n=2):
    source = iter(iterable)
    # one lock keeps the pulls in a single order
    lock = threading.Lock()

    def consumer(cell):
        while True:
            with lock:
                if not cell.filled:
                    # the first reader pulls the value
                    try:
                        cell.value = next(source)
                    except StopIteration:
                        return
                    cell.filled = True
                    cell.next = Cell()
                value, cell = cell.value, cell.next
            yield value
```

```
    # every reader starts at the same first cell
    head = Cell()
    return tuple(consumer(head) for _ in range(n))


if __name__ == "__main__":
    a, b = tee(range(5))
    # readers are independent, at any speed
    assert next(a) == 0 and next(a) == 1
    assert list(b) == [0, 1, 2, 3, 4]
    assert list(a) == [2, 3, 4]
    x, y, z = tee_queues("abc", 3)
    assert list(x) == list(y) == list(z) == list("abc")
    # works on an endless source, which tee_naive cannot
    def count():
        i = 0
        while True:
            yield i
            i += 1
    e1, e2 = tee(count())
    assert [next(e1) for _ in range(3)] == [0, 1, 2]
    assert next(e2) == 0
    assert [list(i) for i in tee_naive([1, 2], 2)] == [
        [1, 2], [1, 2]]
```

NOTES FROM CANDIDATES

The round depends on Python iterator-protocol fluency as much as general data-structure reasoning. Clarify laziness, source exhaustion, invalid n, and whether concurrent consumption is in scope before implementing.
A correct baseline consumes the source lazily once and gives each returned iterator its own queue: when one queue is empty, pull one source value, append it to every queue, then return it from the requesting queue. If n iterators trail across a gap of g values, however, those queues can retain O(n·g) buffered copies. For the optimal shared-state design, keep one linked buffer (or indexed deque) of source values plus one cursor per iterator, and discard a buffered value only after every cursor has passed it. That preserves independent consumption with O(g+n) auxiliary state, where g is the distance between the fastest and slowest consumers.

PREPARATION

Implement a custom Python iterator class from memory, including __iter__ and __next__.
Trace two and three consumers advancing at different speeds and identify what state must remain available.
Compare a fully materialized baseline with a lazy shared-state design and explain the memory behavior.

STUDY PATH: question 145 of 167 (more practice)

Priority score 3.2 | Stacks and queues | asked 132 days ago | frequency Low

MEDIUM | 1Point3Acres | onsite-coding | 2 of 3

Nested Function Expression Evaluator

ASKED 2026-06-10 | 132 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 5** **30 DAYS 0** **PRIORITY 175/228**

ABOUT THIS QUESTION

Topic: More practice - Stacks and queues

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 770 Basic Calculator IV (related)

Frequency: Low | Last asked: 2026-06-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-function-expression-evaluator

QUESTION

EXAMPLES

```
add(1, sub(1, 0)) -> 2
add(2, mul(3, pow(4, 5))) -> 3074
```

APPROACH

Write a small tokenizer for identifiers, integers, commas, and parentheses.
Practice a recursive parseExpr() that returns both the value and the next index.
Drill the variable-arity follow-up by collecting arguments until) instead of assuming exactly two.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which functions exist, whether they take a fixed or variable number of arguments, whether numbers can be negative, how division behaves, and how errors are reported.

BRUTE FORCE TO SAY OUT LOUD

Repeatedly find the innermost call with a regex and replace it with its value. Quick but fragile.

OPTIMAL IDEA

Tokenize into identifiers, integers, commas and parentheses. Write parse(i) returning (value, next index): a number returns itself; a name must be followed by '(', then loop reading arguments until ')', then apply the function. A function table keeps new operators one line each. O(n), recursion depth equals nesting depth.

DRY-RUN THESE CASES

add(1, sub(1, 0)) is 2. add(2, mul(3, pow(4, 5))) is 3074. A bare number. Variable argument counts. Unknown function names.

LIKELY FOLLOW-UPS

Variables, iterative parsing with two stacks, or division semantics.

PITFALLS

INTERVIEW-DAY PLAN

Losing the next index after parsing an argument. Treating '-' in '-5' as an operator.

HOW TO THINK ABOUT IT

An expression with function calls inside function calls has a grammar: expression = number | name '(' expression, expression, ... ')'. Each call can be evaluated only after its arguments, so write a function that parses one expression and reports where it stopped (recursive descent). A tokenizer first, then parse(i) returning (value, next index), keeps the bookkeeping small.

Basis: Recursive descent parsing, in the spirit of LeetCode 224 / 726; closest on LeetCode is 770, Basic Calculator IV.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def evaluate(text):
    while has_call(text):
        call = find_innermost_call(text)
        value = apply(call)  # e.g. add(2, 3)
        text = text.replace(call, str(value))
    return int(text)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every replacement rescans and rebuilds the string, O(n) per call and O(n^2) in total, and it cannot report errors well. A tokenizer plus recursive descent reads the text once, O(n).

OPTIMAL SOLUTION · Tokenizer + recursive descent over name(arg, arg, ...)

TIME	SPACE
O(n)	O(depth)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support variadic functions and variables.

Collect arguments until ')' instead of assuming two (as the code does). For variables, pass an environment dict into parse and treat an identifier that is not followed by '(' as a lookup. Define the error behaviour for undefined names.

FOLLOW-UP 2 How do you make it iterative?

Use two stacks: one of pending function names and one of lists of evaluated arguments. On an identifier push the name and a new list, on a number append to the top list, on ')' pop both, apply the function, and append the result to the new top list. This removes the recursion-depth limit but is easier to get wrong off by one.

FOLLOW-UP 3 What about division?

State the contract: integer or float, truncate toward zero or floor (Python's // floors, which differs for negatives), and divide by zero handling. The implementation above truncates toward zero.

FOLLOW-UP 4 Add variables (an identifier with no parentheses is a lookup).

In parse, check whether the identifier is followed by '('; if not, return env[name]. Decide what an undefined name does (raise, or default to 0).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Nested Function Expression Evaluator
#
# Examples: add(1, sub(1, 0)) -> 2 add(2, mul(3, pow(4,
# 5))) -> 3074
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support variadic functions and variables.
# 2. How do you make it iterative?
# 3. What about division?
# 4. Add variables (an identifier with no parentheses is a
# lookup).
#
# INTERVIEW NOTES
# An expression with function calls inside function calls
# has a grammar: expression = number | name '('
# expression, expression, ... ')'. Each call can be
# evaluated only after its arguments, so write a function
# that parses one expression and reports where it stopped
# (recursive descent). A tokenizer first, then parse(i)
# returning (value, next index), keeps the bookkeeping
# small.
#
# BRUTE FORCE (say it first): Repeatedly replace the
# innermost call with its value
# Find a call whose arguments contain no parentheses (a
# regular expression can do this), compute it, replace the
# text with the result, and repeat until only a number is
# left. It needs no parser, and it is quadratic because
# every replacement rescans and rebuilds the string. Time
# O(n^2). Space O(n).


import math
import re
```

```
# BRUTE FORCE (say it first): Repeatedly replace the
# innermost call with its value | time O(n^2)
_OPS = {
    "add": lambda *a: sum(a),
    "sub": lambda a, b: a - b,
    "mul": lambda *a: math.prod(a),
    "pow": lambda a, b: a**b,
    "div": lambda a, b: int(a / b),
    "min": min,
```

```
    "max": max,
}


# HELPERS for the base question
TOKEN = re.compile(r"-?\d+|[A-Za-z_]\w*|[(),]")


# FOLLOW-UP 3: What about division?
# Answer: State the contract: integer or float, truncate
# toward zero or floor (Python's // floors, which differs
# for negatives), and divide by zero handling. The
# implementation above truncates toward zero.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def divide(a, b):
    # integer division that truncates toward zero
    if b == 0:
        raise ZeroDivisionError("division by zero")
    return int(a / b)


OPS = {
    "add": lambda *a: sum(a),
    "sub": lambda a, b: a - b,
    "mul": lambda *a: math.prod(a),
    "pow": lambda a, b: a ** b,
    "div": divide,
    "min": min,
    "max": max,
}


def evaluate_by_replacement(expr):
    pattern = re.compile(r"([A-Za-z_]\w*)\(([^()]*)\)")
    while True:
        m = pattern.search(expr)
        if not m:
            return int(expr.strip())
        args = [int(x) for x in m.group(2).split(",")]
        expr = (
            expr[: m.start()]
            + str(_OPS[m.group(1)](*args))
            + expr[m.end() :]
        )
```

```
# BASE QUESTION (optimal): Tokenizer + recursive descent
# over name(arg, arg, ...)
# Idea: Tokenize into identifiers, integers, commas and
# parentheses.
def evaluate(expr, env=None):
    env = env or {}
    tokens = TOKEN.findall(expr)

    def parse(i):
        # returns (value, index after this expression)
        tok = tokens[i]
        if re.fullmatch(r"-?\d+", tok):
            return int(tok), i + 1
        # FOLLOW-UP 4: variables come from env
        # a name not followed by "(" is a variable
```

```
        if i + 1 >= len(tokens) or tokens[i + 1] != "(":
            if tok not in env:
                raise ValueError(f"undefined {tok!r}")
            return env[tok], i + 1
        if tok not in OPS:
            raise ValueError(f"unknown function {tok!r}")
        i += 2
        args = []
        # read arguments until the closing bracket
        while tokens[i] != ")":
            value, i = parse(i)
            args.append(value)
            if tokens[i] == ",":
                i += 1
        return OPS[tok](*args), i + 1

    value, end = parse(0)
    if end != len(tokens):
        raise ValueError("trailing tokens")
    return value


# FOLLOW-UP 2: How do you make it iterative?
# Answer: Use two stacks: one of pending function names
# and one of lists of evaluated arguments.
def evaluate_iterative(expr):
    # two stacks: function names and their argument lists
    names, args, tokens = [], [[]], TOKEN.findall(expr)
```

```
    for tok in tokens:
        if re.fullmatch(r"-?\d+", tok):
            args[-1].append(int(tok))
        elif tok == "(":
            # the name just pushed now owns a new list
            args.append([])
        elif tok == ")":
            result = OPS[names.pop()](*args.pop())
            args[-1].append(result)
        elif tok != ",":
            names.append(tok)
    return args[0][0]


if __name__ == "__main__":
    assert evaluate("add(1, sub(1, 0))") == 2
    assert evaluate("add(2, mul(3, pow(4, 5)))") == 3074
    # variadic functions
    assert evaluate("add(1, 2, 3, 4)") == 10
    assert evaluate("max(3, min(9, 4), 1)") == 4
    # variables come from an environment
    assert evaluate("add(x, mul(x, 2))", {"x": 5}) == 15
    # division truncates toward zero
    assert evaluate("div(-7, 2)") == -3
    for bad in ("foo(1)", "add(1", "add(1) 2", "y"):
        try:
            evaluate(bad)
            raise AssertionError(bad)
        except (ValueError, IndexError):
            pass
    # the iterative version agrees and has no depth limit
    big = "add(2, mul(3, pow(4, 5)))"
    assert evaluate_iterative(big) == 3074
    deep = "add(1, " * 3000 + "0" + ")" * 3000
    assert evaluate_iterative(deep) == 3000
```

NOTES FROM CANDIDATES

Tokenizer + recursive descent is the cleanest approach: parse operator, consume (, parse comma-separated child expressions, consume), then evaluate.
Iterative parsing works but is easy to make off-by-one errors under interview pressure; one candidate was specifically flagged for minor syntax and bounds issues.
Clarify division behavior for negative values and non-divisible integers.

PREPARATION

Write a small tokenizer for identifiers, integers, commas, and parentheses.
Practice a recursive parseExpr() that returns both the value and the next index.
Drill the variable-arity follow-up by collecting arguments until) instead of assuming exactly two.

STUDY PATH: question 146 of 167 (more practice)

Priority score 1.78 | Stacks and queues | asked 287 days ago

MEDIUM | PracHub | Technical Screen | 3 of 3

Find longest increasing contiguous subarray

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 146/228

ABOUT THIS QUESTION

Topic: More practice - Stacks and queues

Optimal answer from: the interview site's own solution

LeetCode match: LC 674 Longest Continuous Increasing Subsequence (variant: one modification) (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/find-longest-increasing-contiguous-subarray

QUESTION

Overview: This question evaluates proficiency in array processing and algorithmic optimization, focusing on recognition of contiguous subarray properties and handling a single-value modification to achieve strict monotonicity.

APPROACH

Compute run lengths ending at i (left) and starting at i (right); for each i try changing nums[i] to bridge: left[i-1] + 1 + right[i+1] if nums[i+1] - nums[i-1] >= 2. O(n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether increasing is strict, whether the modified value can be any integer or any real, and whether you must change exactly one element or at most one.

BRUTE FORCE TO SAY OUT LOUD

Try changing each element to every useful value and recompute the run. O(n^2).

OPTIMAL IDEA

Compute left[i], the length of the strictly increasing run ending at i, and right[i], the run starting at i. Changing nums[i] can bridge the neighbors if nums[i + 1] - nums[i - 1] >= 2 (integers), giving left[i - 1] + 1 + right[i + 1]. Otherwise you can only extend one side: left[i - 1] + 1 or right[i + 1] + 1. Take the maximum over all i, and also the run with no change. O(n).

DRY-RUN THESE CASES

n = 1, a strictly decreasing array, a plateau like [1, 2, 2, 3], neighbors that differ by exactly 1 (cannot bridge with integers), the change at the first or last index.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Allow k changes (needs a sliding window with at most k bad positions), allow non-strict increase, or return the subarray bounds.

PITFALLS

Forgetting the integer gap condition. Index errors at the array ends.

HOW TO THINK ABOUT IT

A run that has a single 'bad' spot can be repaired by changing one element, so look at each position i as the element to change. The best run through i joins the increasing run that ends just before it and the one that starts just after it, and the join is legal only if the values leave enough room between the two neighbours (for integers, a[i + 1] - a[i - 1] >= 2).

Basis: LeetCode 674 (Longest Continuous Increasing Subsequence) with one allowed modification. Assumed rule (confirm it): strictly increasing, you may change at most one element to any integer.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    best = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
            part = a[i:j + 1]
            if fixable_with_one_change(part):
                best = max(best, len(part))
    return best
def fixable_with_one_change(part):
    bad = [k for k in range(1, len(part))
           if part[k] <= part[k - 1]]
    if not bad:
        return True
    if len(bad) > 1:
        return False
    return has_room(part, bad[0])
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) subarrays, each tested in O(n): O(n^3). Run lengths ending and starting at each index, then trying each index as the changed element, is O(n).

OPTIMAL SOLUTION · Left and right run lengths, try every position as the changed element

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 No modification allowed (LeetCode 674).

One pass keeping the length of the current strictly increasing run; reset to 1 when a[i] <= a[i - 1]. O(n), O(1).

FOLLOW-UP 2 Up to k modifications.

Use a sliding window over the array that allows at most k positions where the increasing property fails and cannot be repaired; careful with the integer room condition between neighbours. Often handled with DP over (position, changes used).

FOLLOW-UP 3 Values are real numbers, not integers.

Then there is always room between two different neighbours, so the join condition becomes a[i + 1] > a[i - 1] instead of a gap of at least 2.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find longest increasing contiguous
# subarray
#
# This question evaluates proficiency in array processing
# and algorithmic optimization, focusing on recognition of
# contiguous subarray properties and handling a single-
# value modification to achieve strict monotonicity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. No modification allowed (LeetCode 674).
# 2. Up to k modifications.
# 3. Values are real numbers, not integers.
#
# INTERVIEW NOTES
# A run that has a single 'bad' spot can be repaired by
# changing one element, so look at each position i as the
# element to change. The best run through i joins the
# increasing run that ends just before it and the one that
# starts just after it, and the join is legal only if the
# values leave enough room between the two neighbours (for
# integers, a[i + 1] - a[i - 1] >= 2).
#
# BRUTE FORCE (say it first): Test every subarray for
# 'increasing after at most one change'
# For each subarray count the places where an element is
# not larger than its predecessor, and check whether one
# change can fix it (one such spot and the neighbours
# leave room, or none). Exact and slow; the oracle for the
# O(n) method. Time O(n^3). Space O(1).


# BRUTE FORCE (say it first): Test every subarray for
# 'increasing after at most one change' | time O(n^3)
def longest_one_change_brute(a):
    n = len(a)
    best = 0
    for i in range(n):
        for j in range(i, n):
            sub = a[i : j + 1]
            ok = any(
                all(
                    x[k] < x[k + 1]
                    for k in range(len(x) - 1)
                )
                for x in (
                    [sub]
                    + [
                        sub[:k] + [v] + sub[k + 1 :]
                        for k in range(len(sub))
                        for v in range(
                            min(a) - len(a) - 2,
                            max(a) + len(a) + 3,
                        )
                    ]
                )
            )
            if ok:
                best = max(best, len(sub))
    return best


# BASE QUESTION (optimal): Left and right run lengths, try
# every position as the changed element
# Idea: left[i] is the length of the strictly increasing
# run ending at i and right[i] the length of the one
# starting at i.
def longest_one_change(a, integers=True):
    # strictly increasing after changing AT MOST one value
    n = len(a)
    if n == 0:
        return 0
    # left[i]: run ending at i; right[i]: run starting at
    # i
    left, right = [1] * n, [1] * n
    for i in range(1, n):
        if a[i] > a[i - 1]:
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if a[i] < a[i + 1]:
            right[i] = right[i + 1] + 1
    # the gap needed between two neighbours: 2 for whole
    # numbers (one number must fit between), > 0 for reals
    best = max(left)
    for i in range(n):
        # change a[i] to join the run on its left ...
        if i > 0:
            best = max(best, left[i - 1] + 1)
        # ... or the run on its right ...
        if i < n - 1:
            best = max(best, right[i + 1] + 1)
        # ... or both, when a value fits between the
        # neighbours
        if 0 < i < n - 1:
            room = a[i + 1] - a[i - 1]
            fits = room >= 2 if integers else room > 0
            if fits:
                joined = left[i - 1] + 1 + right[i + 1]
                best = max(best, joined)
    return min(best, n)


# FOLLOW-UP 1: No modification allowed (LeetCode 674).
# Answer: One pass keeping the length of the current
# strictly increasing run; reset to 1 when a[i] <= a[i -
# 1]. O(n), O(1).
def longest_increasing(a):
    # LeetCode 674: no change allowed, one pass
    best = run = 0
    for i, x in enumerate(a):
        run = run + 1 if i and a[i - 1] < x else 1
        best = max(best, run)
    return best
```

```
if __name__ == "__main__":
    assert longest_increasing([1, 3, 5, 4, 7]) == 3
    assert longest_increasing([]) == 0
    assert longest_one_change([1, 3, 5, 4, 7]) == 5
    # equal neighbours leave no room for a whole number
    assert longest_one_change([1, 2, 2, 3]) == 3
    nums = [1, 2, 2, 3]
    reals = longest_one_change(nums, integers=False)
    assert reals == 4
    assert longest_one_change([5, 4, 3, 2, 1]) == 2
    assert longest_one_change([7]) == 1
    assert longest_one_change([]) == 0
    # never more than the array length
    assert longest_one_change([1, 2, 3]) == 3
```

FUNCTION AND TEST CASES

Signature

```
def longestIncreasing(nums: list[int]) -> int:
```

Sample call

```
longestIncreasing([1, 2, 2, 3, 4])
```

Test cases

Input	Expected
`([1, 2, 2, 3, 4],)`	`3`
`([1, 2, 3, 4, 5],)`	`5`